СОДЕРЖАНИЕ

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ 3

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ 4

ВВЕДЕНИЕ 5

ГЛАВА 1 7

1.1 Анализ существующих курсов по машинному обучению 7

1.1.1 Машинное обучение на ФКН НИУ ВШЭ 7

1.1.2 Машинное обучение на ВМК МГУ 8

1.1.3 Deep Learning School МФТИ 8

1.1.4 Open ML Course: Классические модели ML 9

1.1.5 Курс машинного обучения girafe-ai 9

1.1.6 Karpov.Courses: Инженер машинного обучения 10

1.1.7 Яндекс Практикум: ML-инженер с опытом 10

1.1.8 Обобщение результатов анализа 11

1.2 Анализ инструментов для автоматической проверки 13

1.2.1 nbgrader 14

1.2.2 Otter-Grader 14

1.2.3 CodeGrade 15

1.2.4 Яндекс.Контест 15

1.2.5 Stepik 15

1.2.6 GitHub Classroom 16

1.2.7 Kaggle InClass 17

1.2.8 PyEvalAI 17

1.2.9 Обобщение результатов анализа 18

1.3 Формулировка гипотезы и образ решения 19

1.3.1 Формулировка гипотезы 19

1.3.2 Образ решения 20

ГЛАВА 2 22

2.1 Рабочая программа дисциплины 22

2.1.1 Пререквизиты 22
2.1.2 Разделы 22
2.1.3 Темы 22
2.1.4 Оцениваемые задания 23
2.1.5 Результаты обучения 25
2.2 Внедрение автоматизированной проверки в nbgrader 25
2.2.1 Требования к работе 26
2.2.2 Реализация расширения nbgrader 27
2.2.3 Создание задания преподавателем 27
2.2.4 Генерация студенческой версии 28
2.2.5 Конфигурация и запуск проверки 28
2.3 Создание системы для проверки проектов 29

ГЛАВА 3 30

ЗАКЛЮЧЕНИЕ 31

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 31

А РУКОВОДСТВО ПО ИСПОЛЬЗОВАНИЮ ШАБЛОНА 34

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ

БЯМ — большая языковая модель

ВМК — факультет вычислительной математики и кибернетики

ДЗ — домашнее задание

ИИ — искусственный интеллект

КР — контрольная работа

МО — машинное обучение

МГУ — Московский государственный университет

МФТИ — Московский физико-технический институт

НИУ ВШЭ — Национальный исследовательский университет «Высшая школа экономики»

ФКН — факультет компьютерных наук

CV — Computer Vision (компьютерное зрение)

CLI — Command Line Interface (командная строка)

HTML — Hyper Text Markup Language (язык разметки гипертекста)

DLS — Deep Learning School

GPU — графический процессор

JN — Jupyter Notebook

LMS — Learning Management System (система управления обучением)

ML — Machine Learning (машинное обучение)

ODS — Open Data Science

NLP — Natural Language Processing (обработка естественного языка)

RecSys — recommender systems (рекомендательные системы)

SQL — Structured Query Language (язык структурированных запросов)

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

Автоматическая проверка — оценивание учебных заданий программной системой без непосредственной ручной проверки преподавателем

Большая языковая модель (БЯМ) — нейросетевая модель, генерирующая текст

Кросс-проверка — взаимная проверка работ обучающимися по заданным критериям

Ноутбук — файл с расширением .ipynb для работы в Jupyter Notebook

Открытый теоретический вопрос — задание, требующее развёрнутого текстового ответа с объяснением, интерпретацией или выводом

Юнит-тест — автоматическая проверка отдельного фрагмента программы по заранее заданным условиям

Ячейка — блок кода в Jupyter Notebook, который может быть выполнен и проверен

Jupyter Notebook — интерактивная вычислительная среда, объединяющая код, текстовые пояснения и результаты выполнения в одном документе

ВВЕДЕНИЕ

Машинное обучение за последнее десятилетие стало одной из ключевых компетенций для IT-специалистов. Спрос на образовательные программы по данному направлению неуклонно растет: курсы предлагают ведущие университеты (НИУ ВШЭ, МГУ, МФТИ), корпоративные школы (ШАД Яндекса, Т-Образование) и открытые платформы (ODS, Stepik, Karpov.Courses). Вместе с ростом числа слушателей обостряется проблема масштабируемости оценивания: преподаватели тратят значительную часть времени на проверку работ, а при увеличении потока студентов качество обратной связи снижается.

Анализ существующих курсов по машинному обучению показал, что автоматизация оценивания остается фрагментарной. Существующие инструменты - юнит-тесты в Jupyter Notebooks, контестные системы, тесты с выбором ответа - эффективны для проверки кода и формализованных ответов, но не позволяют оценивать задания открытого типа: теоретические вопросы, развернутые объяснения, интерпретацию результатов. Такие задания по-прежнему проверяются вручную, что создает узкое место в образовательном процессе. Появление больших языковых моделей (БЯМ) открывает возможность автоматизировать проверку заданий, требующих понимания естественного языка и оценки содержательной корректности ответов.

Новизна работы заключается в разработке и апробации формата образовательного курса по машинному обучению, в котором автоматическая оценка открытых ответов реализована с использованием БЯМ, интегрированного в существующий учебный пайплайн nbgrader. В отличие от решений, ориентированных только на проверку кода и закрытых тестов, предложенный подход объединяет новый формат курса, встроенный БЯМ и его применение на реальной аудитории студентов.

Практическая значимость работы определяется внедрением разработанного курса в образовательный процесс НИУ ВШЭ, а также апробацией системы автоматической проверки на реальной аудитории студентов, что позволяет оценить её применимость и эффективность в условиях учебного процесса и

потенциальную переносимость на другие технические дисциплины.

Целью данной работы является разработка автоматизированной системы проверки учебных заданий на основе больших языковых моделей для курса по машинному обучению, ее интеграция в образовательный процесс и апробация.

Для достижения данной цели были поставлены следующие задачи:

1) Провести анализ существующих курсов по машинному обучению и сформировать профиль целевой аудитории.
2) Исследовать подходы к автоматизированной проверке учебных заданий в образовательной среде.
3) Разработать методологию преподавания и структуру учебного курса по машинному обучению.
4) Разработать контрольные задания, критерии оценивания и учебно-методические материалы курса.
5) Реализовать автоматизированную систему проверки учебных заданий с использованием больших языковых моделей.
6) Провести апробацию разработанного курса и системы автоматической проверки в образовательном процессе НИУ ВШЭ и проанализировать полученные результаты.

Критериями успешности решения выступают: сокращение времени проверки учебных заданий, высокая степень согласованности автоматических оценок с экспертными, а также положительная оценка скорости и качества обратной связи со стороны студентов по итогам апробации.

Работа состоит из введения, трех глав и заключения. В первой главе описана предметная область, проведен анализ существующих курсов и подходов к автоматизированной проверке, сформулированы требования к решению. Во второй главе представлена архитектура разработанного курса и системы автоматической проверки, описаны методология, структура заданий и механизм оценивания на основе БЯМ. В третьей главе приведены результаты апробации курса, анализ метрик качества автоматической проверки и обратная связь от студентов.

ГЛАВА 1

1.1 Анализ существующих курсов по машинному обучению

Для формирования методологической и содержательной основы разрабатываемого курса был проведён анализ семи образовательных программ по машинному обучению. Поскольку область активно развивается, при отборе приоритет отдавался актуальным и востребованным программам с открытыми материалами, допускающими содержательный анализ системы контроля знаний.

Выборка охватывает три сегмента. Первый — академические курсы ведущих российских университетов (НИУ ВШЭ, МГУ, МФТИ): они задают стандарт теоретической глубины и отражают опыт, с которым сталкивается студент профильной специальности. Второй — открытые образовательные инициативы сообществ (ODS, girafe-ai): они представляют альтернативу университетскому образованию для широкой аудитории без формального отбора. Третий — коммерческие онлайн-платформы (Karpov.Courses, Яндекс.Практикум): они отражают запросы рынка труда и ориентированы на практическую подготовку специалистов.

Для каждого курса рассматривались формат обучения, содержание программы, система контроля знаний и степень автоматизации процесса оценивания. Все анализируемые курсы ведутся на русском языке.

1.1.1 Машинное обучение на ФКН НИУ ВШЭ

Курс Е. А. Соколова [1] является одним из наиболее известных академических курсов по машинному обучению. Обучение проходит в очном формате для бакалавров факультета компьютерных наук НИУ ВШЭ, при этом все материалы, включая конспекты лекций и видеозаписи 2021 года, опубликованы в открытом доступе. Программа охватывает классические разделы машинного обучения.

Система контроля включает теоретические и практические домашние задания, контрольную работу и экзамен. Теоретические домашние задания и контрольные работы включают открытые вопросы с объяснением, а практические подразумевают работу в Jupyter Notebook. Домашние задания

принимаются через платформу Anytask - она может только собирать ответ студентов, при этом не поддерживает автоматизацию проверки.
Практические задания в Jupyter Notebook частично автоматизированы юнит-тестами, однако все практические и теоретические работы проверяют ассистенты. Ручная проверка таких заданий фиксируется как ключевое ограничение: при росте потока студентов она масштабируется только увеличением количества ассистентов.

1.1.2 Машинное обучение на ВМК МГУ

Курс машинного обучения для студентов третьего курса факультета вычислительной математики и кибернетики МГУ [2] по содержанию близок к программе НИУ ВШЭ. Обучение организовано в очном формате и сопровождается практическими и теоретическими видами контроля. В практические входит работа в Jupyter Notebook и соревнования на платформе Kaggle, а в теоретические - коллоквиумы, тесты на лекциях, открытые вопросы в лабораторных работах и экзамен.
Проверка практических заданий автоматизирована при помощи юнит-тестов, при этом некоторые задания требуют кросс-проверки между студентами в количестве трех человек [3].
Использование автотестов демонстрирует реализуемость автоматической проверки кода прямо внутри Jupyter Notebook. Кросс-проверка открытых ответов тремя студентами, напротив, отмечается как неудовлетворительный подход: она не обеспечивает согласованности оценок и перекладывает нагрузку на самих студентов.

1.1.3 Deep Learning School МФТИ

Школа глубокого обучения (DLS) при МФТИ [4] является бесплатным онлайн-курсом, реализованным на платформе Stepik. Программа разделена на четыре части. В первой части обсуждаются основы машинного обучения и нейронных сетей, во второй части - NLP, третья посвящена CV, а четвертая аудио.
Оценивание включает домашние задания в Jupyter Notebook, тесты, соревнования на платформе Kaggle и итоговый проект. Тесты автоматизированы на платформе Stepik. Домашние задания проверяются в формате кросс-валидации между студентами. Проект проверяется

преподавателем, закрепленным за студентом.

Курс имеет платный тариф, в котором предлагается доступ в приватный чат, персональный ментор, приоритетная проверка домашних заданий и дополнительные вебинары.

Кросс-проверка домашних заданий является ограничением — по тем же причинам, что и в курсе МГУ, при этом платный тариф показывает, что такой подход проверки может быть долгим. Также курс полностью не автоматизирован тестами и кросс-проверками: проект вручную проверяется преподавателем, а для более расширенной обратной связи необходимо перейти на платный тариф – это показывает важность обратной связи и подкрепляет сложность масштабирования на большую аудиторию студентов.

1.1.4 Open ML Course: Классические модели ML

Курс сообщества Open Data Science [5] представляет собой полностью онлайн-формат обучения и распространяется бесплатно без отбора слушателей. Темы включают линейные модели, решающие деревья и дополнительные главы по машинному обучению. Длительность курса составляет два месяца.

Проверочные контроли состоят из тестов, дополнительно дается возможность пройти соревнование. Проверка заданий полностью автоматизирована на собственной платформе ODS.ai.

Полная автоматизация проверки достигается здесь за счёт отказа от заданий открытого типа — контроль сводится к тестам и соревнованиям, поэтому этот компромисс фиксируется как нежелательный.

1.1.5 Курс машинного обучения girafe-ai

Открытый курс girafe-ai [6] содержит полный цикл изучения машинного обучения: от линейных моделей и метода опорных векторов до нейронных сетей, алгоритма обратного распространения ошибки, эмбеддингов и архитектур seq2seq. Материалы курса и видеозаписи лекций опубликованы в открытом доступе.

Контроль знаний организован через серию заданий и лабораторных работ в Jupyter Notebook, а также итоговый экзамен. Некоторые задания сразу включают тесты для самопроверки, а некоторые содержат теоретические вопросы. Нигде явно не описано, как именно и кем проверяются работы.

Теоретические вопросы и итоговый экзамен воспроизводят модель академических курсов и дополнительно подтверждают их подход: открытые задания необходимы для диагностики понимания алгоритмов, а не только их практического применения.

1.1.6 Karpov.Courses: Инженер машинного обучения

Коммерческий курс [7] реализован полностью в онлайн-формате. Программа ориентирована на подготовку специалистов от базового уровня до внедрения моделей в производственную эксплуатацию: линейные модели, деревья решений, нейронные сети, обработка естественного языка, рекомендательные системы, A/B-тестирование и развертывание моделей.

Классические семинары в курсе отсутствуют, практическая подготовка обеспечивается за счет решения прикладных задач на собственной платформе и разработки итогового проекта в формате ML-сервиса с проверкой экспертов.

Курс демонстрирует, что проектная модель оценивания с развёрнутой обратной связью является ключевой для подготовки специалистов. Полное отсутствие теоретических открытых вопросов рассматривается как ограничение: такой подход не позволяет диагностировать глубину понимания алгоритмов, что становится ограничением для академического курса.

1.1.7 Яндекс Практикум: ML-инженер с опытом

Курс «ML-инженер с опытом» от Яндекс Практикума [8] реализован полностью в онлайн-формате и рассчитан на четыре месяца обучения. Программа ориентирована на специалистов с базовыми знаниями классического машинного обучения и охватывает полный производственный цикл.

Система оценивания строится на пяти учебных проектах и итоговом проекте, выполняемом в условиях, приближённых к реальным рабочим задачам. Проверка проектов осуществляется ревьюерами с предоставлением развёрнутой письменной обратной связи. Практика выполняется во встроенной инфраструктуре платформы с доступом к Yandex Cloud. Про теоретические задания отдельно ничего не говорится. Студентам доступен Практикум ИИ - нейросеть, отвечающая на вопросы по материалу курса.

Курс подтверждает масштабируемость проектной модели в коммерческом сегменте, как и Karpov.Courses.

1.1.8 Обобщение результатов анализа

Результаты анализа систематизированы в трёх таблицах: в таблице 1.1 — общие характеристики и тематическое содержание программ, в таблице 1.2 — виды оценочных мероприятий, в таблице 1.3 — организация проверки учебных заданий.

Таблица 1.1 — Общая характеристика и тематическое содержание курсов

Курс	Тип	Формат	Год	Кл. МО	Гл. об.	Внедр.	Доп. темы
МО ФКН НИУ ВШЭ	Академический	Очный	2026	Да	—	—	—
МО ВМК МГУ	Академический	Очный	2026	Да	—	—	—
Open ML Course	Открытый	Онлайн	2026	Да	—	—	—
DLS МФТИ	Открытый	Онлайн	2026	Да	Да	—	NLP, CV, аудио
girafe-ai	Открытый	Очный	2021	Да	Да	—	seq2seq, эмбеддинги
Karpov.Courses	Коммерческий	Онлайн	2026	Да	Да	Да	A/B тесты
Яндекс Практикум	Коммерческий	Онлайн	2025	—	—	Да	RecSys, NLP
Кл. МО — классическое МО; Гл. об. — глубинное обучение; Внедр. — внедрение моделей в производство; «—» — отсутствует.							

Таблица 1.2 — Виды оценочных мероприятий

Курс	Практ. задания	Теор. задания	Тесты	Экз./КР	Проект	Соревн.
МО ФКН НИУ ВШЭ	Да (JN)	Да	Да	Да	—	—
МО ВМК МГУ	Да (JN)	Да	Да	Да	—	Да
Open ML Course	—	—	Да	—	—	Да
DLS МФТИ	Да (JN)	Да	Да	—	Да	Да
girafe-ai	Да (JN)	Да	Да	Да	—	—
Karpov.Courses	Да (плат.)	—	—	—	Да	—
Яндекс Практикум	Да (плат.)	—	—	—	Да	—
JN — Jupyter Notebook; плат. — собственная платформа курса; Соревн. — соревнования.						

Таблица 1.3 — Организация проверки домашних заданий

Курс	Платформа ДЗ	Автотесты	Кросс-пров.	Ручная пров.
МО ФКН НИУ ВШЭ	Anytask	—	—	Да
МО ВМК МГУ	Собственная	Да	Да	Да
Open ML Course	ODS.ai	Да	—	—
DLS МФТИ	Stepik, Kaggle	Да	Да	Да
girafe-ai	GitHub	Да	—	—
Karpov.Courses	Собственная	Да	—	Да
Яндекс Практикум	Собственная	—	—	Да

Проведённый анализ позволяет сформулировать ряд ключевых наблюдений. **Jupyter Notebook как стандарт практических заданий.** Четыре из семи рассмотренных курсов используют Jupyter Notebook в качестве основной

среды выполнения практических заданий. В ряде программ код в ноутбуках уже проверяется автоматически посредством юнит-тестов, а ответы на теоретические вопросы и выводы необходимо выписать в ячейках JN, что подтверждает принципиальную осуществимость автоматизации в данной среде и обозначает Jupyter Notebook как естественную точку интеграции для расширенной системы оценивания.

Классическое машинное обучение и теоретические задания. Классические алгоритмы машинного обучения образуют обязательное ядро всех семи программ вне зависимости от их типа и формата. В академических курсах изучение классических алгоритмов неизменно сопровождается теоретическими заданиями открытого типа — вопросами с развёрнутым ответом, интерпретацией результатов, формулировкой выводов, — которые направлены на диагностику глубины понимания математической природы алгоритмов, а не только умения применять их на практике. Задания открытого типа присутствуют в четырех из семи программ и характерны прежде всего для академических и открытых курсов.

Итоговый проект как маркер практической ориентации. Проект предусмотрен в трёх из семи курсов: в двух коммерческих (Karpov.Courses, Яндекс Практикум) и одном открытом (DLS МФТИ). Примечательно, что именно коммерческие программы последовательно выстраивают обучение вокруг проекта как сквозной формы работы — это отражает запрос рынка труда на специалистов, способных решать комплексные задачи в производственных условиях. Вместе с тем оценивание проекта требует развёрнутой индивидуальной обратной связи от преподавателей или ревьюеров, что при увеличении числа студентов существенно нагружает преподавательский состав.

Проверка открытых заданий как ключевое узкое место. Оценивание теоретических заданий открытого типа сводится к трём подходам: ручная проверка преподавателем, кросс-проверка между студентами или исключение подобных заданий из системы контроля. Каждый из них имеет существенные ограничения: ручная проверка не масштабируется при росте потока студентов, кросс-проверка не обеспечивает согласованности оценивания и перекладывает нагрузку на самих обучающихся, а отказ от

открытых заданий снижает глубину диагностики знаний. Таким образом, автоматизация охватывает проверку кода и формализованных ответов, но практически не применяется к оценке текстовых ответов, требующих содержательного анализа. Это противоречие определяет необходимость рассмотрения существующих инструментов автоматической проверки.

1.2 Анализ инструментов для автоматической проверки

Задача автоматизации проверки учебных заданий решается различными инструментами: от классических систем юнит-тестирования Jupyter Notebook до коммерческих платформ и исследовательских прототипов на основе больших языковых моделей.

Для систематического сравнения введём критерии, существенные с точки зрения настоящей работы.

- **Поддержка Jupyter Notebook.** Анализ в 1.1 показал, что Jupyter Notebook является стандартом для практических заданий в курсах по машинному обучению, поэтому его поддержка является важным критерием.
- **Наличие открытых заданий.** Инструмент должен поддерживать создание открытых заданий, требующих развёрнутого текстового ответа с объяснением, интерпретацией или выводом.
- **Наличие автотестов.** Наличие механизмов юнит-тестирования кода или сверки с фиксированным правильным ответом.
- **Открытость кода.** Доступность исходного кода для изучения, модификации и расширения.
- **Интеграция с LMS.** Возможность встроить инструмент в существующий рабочий процесс преподавателя без значительных изменений инфраструктуры: поддержка CLI и/или отдельные LMS-интеграции.
- **Требования к инфраструктуре.** Необходимые аппаратные и программные ресурсы для развёртывания: наличие GPU, серверных мощностей и т. п.
- **Веб-интерфейс.** Наличие веб-интерфейса для просмотра результатов проверки и обратной связи с преподавателем оценивает порог входа для преподавателя.

Ниже рассмотрены восемь наиболее распространённых решений. Для каждого приведены краткая характеристика и явный вывод о применимости

к автоматизации открытых заданий.

1.2.1 nbgrader

nbgrader — открытая система для создания и проверки заданий в формате Jupyter Notebook, разработанная в UC Berkeley и поддерживаемая сообществом Jupyter [9; 10]. Поддерживает два типа ячеек: автоматически проверяемые ячейки кода на основе assert-тестов и ячейки с открытым ответом. Открытые ответы — теоретические вопросы, текстовые объяснения — проверяются вручную через веб-интерфейс Jupyter Hub и расширения Formgrader. Проверка ячеек кода поддерживает скрытые тесты, которые не видны студенту.

Преподавателю необходимо разметить ячейки в Jupyter специальными комментариями, написать тесты и настроить структуру каталогов. Инструмент доступен как в JupyterHub, так и как консольная утилита – последнее позволяет интегрировать инструмент в существующий рабочий процесс преподавателя.

Отдельным плюсом инструмента является генерация отзыва студенту по результатам проверки в формате HTML. Для автоматически проверяемых ячеек он отображает только балл, а для открытых ответов помимо балла также и отзыв преподавателя.

Код открытый и библиотека активно поддерживается сообществом, что позволяет внедрять новую функциональность и исправлять найденные ошибки. Хотя кодовые ячейки все еще можно оценивать автоматически, проверка открытых ответов остается на ручном уровне, что создает дополнительные трудозатраты на преподавателя.

1.2.2 Otter-Grader

Otter-Grader — открытая система автоматической проверки на Python и R [11; 12]. Как и nbgrader, использует разметку ноутбука специальными комментариями. Ключевое отличие — модель исполнения: команда `otter assign` генерирует из исходного ноутбука студенческую версию и архив для запуска проверки в изолированном Docker-контейнере, что упрощает масштабирование. Открытые вопросы помечаются как `manual: true` и экспортируются в PDF для ручной проверки во внешней системе. Собственного интерфейса для оценки открытых ответов нет.

Код открытый, однако сообщество и документация заметно меньше, чем у nbgrader. Зависимость от внешней системы при проверке открытых ответов означает, что автоматизация этого сценария в рамках самого инструмента не предусмотрена и потребовала бы значительной доработки.

1.2.3 CodeGrade

CodeGrade — коммерческая платформа, разработанная в Университете Амстердама [13]. Имеет нативную поддержку Jupyter Notebook, позволяющую визуально настраивать блоки тестов: проверки ввода-вывода, юнит-тесты, линтеры, структурные проверки и пользовательские скрипты. Интегрируется с LMS и включает встроенную проверку на плагиат. ИИ-ассистент для студентов, помогающий в решении задач, находится в стадии бета-тестирования. Код закрытый; продукт распространяется по подписке.

Открытые текстовые ответы автоматически не оцениваются — проверка остаётся ручной. Коммерческая модель и закрытый код исключают возможность расширения функциональности, что делает инструмент неприменимым для целей настоящей работы.

1.2.4 Яндекс.Контест

Яндекс.Контест — проприетарная платформа, реализующая классическую модель онлайн-судьи (stdin/stdout) [14]. Поддерживает программные задачи с тестами и пользовательские проверки. Jupyter Notebook не поддерживается; открытые теоретические вопросы оцениваться не могут.

Преподавателю необходимо подготовить условие задачи и набор тестов; настройка выполняется через веб-интерфейс платформы. Инструмент хорошо знаком студентам технических специальностей и активно применяется в олимпиадной практике.

Исходный код закрытый, платформа ориентирована исключительно на алгоритмические задачи. По этим причинам инструмент не применим в контексте настоящей работы.

1.2.5 Stepik

Stepik — образовательная платформа, поддерживающая 18 типов автоматически проверяемых заданий: тесты с выбором ответа, численные задачи, проверку формул через SymPy, регулярные выражения, исполнение

кода против тестов, SQL [15]. Для открытых ответов доступна кросс-валидация между студентами или проверка командой курса. Jupyter Notebook не поддерживается.

Преподаватель настраивает задания через веб-интерфейс; для нестандартных проверок можно написать функцию `check(reply)` на Python. Платформа предоставляет встроенную аналитику по результатам студентов и поддерживает сертификацию.

Исходный код закрытый. Jupyter Notebook не поддерживается — код выполняется во встроенном редакторе. Открытые ответы автоматически не оцениваются: кросс-валидация не обеспечивает согласованности оценок и перекладывает нагрузку на студентов, а ручная проверка командой курса не масштабируется. Отсутствие поддержки Jupyter Notebook в совокупности с невозможностью автоматической оценки открытых ответов делают Stepik неприменимым для целей настоящей работы.

1.2.6 GitHub Classroom

GitHub Classroom — бесплатный сервис для управления учебными заданиями на основе Git и GitHub Actions [16]. Каждое задание представлено шаблонным репозиторием; при каждом `git push` в Linux-окружении GitHub Actions запускаются преподавательские тесты, результаты которых отображаются в интерфейсе репозитория. Jupyter Notebook нативно не поддерживается, однако ноутбук может быть преобразован в Python-скрипт.

Преподавателю необходимо подготовить шаблонный репозиторий с тестами и настроить GitHub Actions; от студентов требуется базовое владение Git. Платформа удобно интегрируется с рабочим процессом разработки и приучает студентов к практике контроля версий, что особенно полезно для проектной работы.

Код инфраструктуры открытый. Jupyter Notebook нативно не поддерживается — ноутбук необходимо преобразовывать в Python-скрипт для проверки. Открытые теоретические вопросы платформа не оценивает: проверка строится исключительно на автоматических тестах. Отсутствие нативной поддержки Jupyter и механизма оценки открытых ответов исключают использование инструмента в качестве основного решения.

1.2.7 Kaggle InClass

Kaggle InClass — режим закрытых учебных соревнований, позволяющий преподавателю создать соревнование с приватным тестовым набором [17]. Студенты загружают файлы предсказаний, которые автоматически оцениваются по выбранной метрике с публикацией результатов на лидерборде. Ни код, ни теоретические ответы платформа не анализирует.

Преподавателю необходимо подготовить датасет, разметку, скрытую часть ground truth и описание метрики.

Исходный код закрытый. Ни код решения, ни теоретические ответы платформа не анализирует — оценивается только качество предсказаний. Jupyter Notebook не поддерживается. Инструмент может служить полезным дополнением к основной системе оценивания, однако не решает задачу проверки открытых ответов и не может использоваться как самостоятельное решение.

1.2.8 PyEvalAI

PyEvalAI — закрытый исследовательский прототип, находящийся в стадии апробации [18]. Система оценивает как код, так и текстовые ячейки в Jupyter Notebook, комбинируя юнит-тесты с локально развёрнутой открытой БЯМ. Модель получает на вход условие задачи, эталонное решение, ответ студента и результаты тестов, после чего генерирует численную оценку и персонализированную обратную связь.

Преподавателю необходимо подготовить ноутбуки с заданиями и эталонными ответами, а также развернуть сервер с GPU для работы языковой модели. Инструмент доступен только через веб-интерфейс, консольная утилита отсутствует.

PyEvalAI в отличие от других инструментов поддерживает и автоматическую проверку кодовых ячеек с помощью тестов, и проверку открытых ответов с помощью БЯМ. Однако закрытый код не позволяет изучить, расширить или адаптировать систему под конкретный учебный процесс, а требование GPU-сервера создаёт высокий технический порог входа и делает инструмент неприменимым там, где подобная инфраструктура недоступна. Также отсутствие CLI интерфейса исключает возможность интеграции с другими инструментами преподавателя.

1.2.9 Обобщение результатов анализа

Как видно из таблицы 1.4, все рассмотренные инструменты поддерживают автоматическую проверку закрытых заданий: в разных формах реализованы юнит-тесты для кода, сверка с эталонным ответом и соревновательная оценка предсказаний. Однако при переходе к открытым теоретическим ответам возможности большинства решений резко сужаются: семь из восьми инструментов либо перекладывают такую проверку на преподавателя или студентов через кросс-валидацию, либо вовсе не поддерживают этот тип заданий. Аналогичный вывод следует и по критерию поддержки Jupyter Notebook: нативная интеграция присутствует только у половины рассмотренных систем, тогда как в остальных случаях требуется преобразование ноутбука в скрипт или отказ от этого формата.

Таблица 1.4 — Сравнение инструментов автоматической проверки по базовым критериям

Инструмент	**JN**	**Откр. ответы**	**Автотесты**	**Открытый код**
nbgrader	Нативная	Только вручную	Да	Да
Otter-Grader	Нативная	Только вручную	Да	Да
CodeGrade	Нативная	Только вручную	Да	Нет
PyEvalAI	Нативная	Да, через БЯМ	Да	Нет
Яндекс.Контест	Нет	Нет	Да	Нет
Stepik	Нет	Вручную + кросс-валидация	Да	Нет
GitHub Class-room	Нет	Нет	Да	Нет
Kaggle InClass	Нет	Нет	Да	Нет

Единственным инструментом, сочетающим нативную Jupyter-интеграцию с автоматической оценкой открытых ответов через БЯМ, является PyEvalAI. Однако данные таблицы 1.5 показывают, что это преимущество сопровождается существенными ограничениями: система не интегрируется с LMS и требует более сложной инфраструктуры, включая GPU-сервер. Дополнительно непубликуемый исходный код делает её непригодной в

качестве основы для адаптации под конкретный учебный процесс. Поэтому PyEvalAI подтверждает принципиальную реализуемость подхода, но не может служить готовым решением.

Наиболее подходящей основой для расширения оказывается nbgrader. По базовым критериям он сочетает открытый исходный код, нативную поддержку Jupyter Notebook и автоматическую проверку программных заданий, а по критериям интеграции допускает как работу через CLI, удобную для встраивания в другие системы, включая академические LMS, так и использование веб-интерфейса Formgrader без обязательной LMS-интеграции. Таким образом, единственный незакрытый для nbgrader сценарий - автоматическая оценка открытых теоретических вопросов - не требует замены всей инфраструктуры: достаточно добавить новый тип ячейки с проверкой через БЯМ, сохранив привычный рабочий процесс преподавателя.

Таблица 1.5 — Сравнение инструментов автоматической проверки по критериям интеграции

Инструмент	**Интеграция с LMS**	**Веб-интерфейс**	**Требования к инфраструктуре**
nbgrader	Возможна через CLI	Да	Низкие
Otter-Grader	Возможна через CLI	Нет	Низкие
CodeGrade	Возможна	Да	Отсутствуют
PyEvalAI	Отсутствует	Да	Средние (GPU)
Яндекс.Контест	Отсутствует	Да	Отсутствуют
Stepik	Возможна	Да	Отсутствуют
GitHub Class-room	Возможна	Да	Отсутствуют
Kaggle InClass	Отсутствует	Да	Отсутствуют

1.3 Формулировка гипотезы и образ решения

1.3.1 Формулировка гипотезы

На основании проведённого анализа формулируется следующая гипотеза: интеграция большой языковой модели в систему проверки заданий на базе nbgrader позволит автоматизировать оценку открытых теоретических вопросов в среде Jupyter Notebook, обеспечив при этом качество и

согласованность обратной связи, сопоставимые с ручной проверкой преподавателя, и не требуя от него дополнительных временных затрат на рутинное оценивание. Помимо этого, применение БЯМ для проверки итоговых проектов позволит обеспечить студентам персонализированную развёрнутую обратную связь по нетиповым открытым заданиям.

Проверка гипотезы предполагает разработку соответствующего программного решения, создание курса по классическому машинному обучению и проведение апробации на реальной учебной аудитории.

1.3.2 Образ решения

Предлагаемое решение состоит из двух взаимосвязанных компонентов: расширения системы nbgrader с поддержкой автоматической проверки открытых ответов через БЯМ и системы проверки итогового проекта курса.

Первый компонент представляет собой расширение стандартного функционала nbgrader. В текущем виде инструмент поддерживает два типа ячеек: автоматически проверяемые ячейки с кодом на основе assert-тестов и ячейки с открытым ответом, проверяемые вручную. Предлагается ввести третий тип — ячейки с открытым ответом, оцениваемые автоматически с привлечением БЯМ. Для каждой такой ячейки преподаватель указывает условие задания, эталонный ответ и критерии оценивания. При проверке модель получает эти данные вместе с ответом студента и возвращает числовую оценку и содержательную обратную связь. Такой подход позволяет автоматизировать оценку коротких теоретических вопросов, формулировок выводов и интерпретаций результатов — тех типов заданий, которые принципиально важны для глубокого понимания материала, но традиционно требуют ручной проверки. Интеграция выполняется в рамках существующего интерфейса nbgrader, что не требует от преподавателей изменения привычного рабочего процесса.

Второй компонент — система проверки итогового проекта. В отличие от стандартных заданий проект представляет собой развёрнутую самостоятельную работу студента: постановку задачи, анализ данных, обучение и сравнение моделей, интерпретацию результатов и выводы. Проект сдаётся с кодом и текстовыми пояснениями в открытом виде. БЯМ анализирует как программную, так и текстовую части работы: корректность реализации, качество анализа, обоснованность выводов и

полноту изложения. По результатам анализа модель формирует развёрнутую персонализированную обратную связь по каждому разделу работы и дает оценку. Данный подход позволяет предоставить студенту детальный разбор его работы без привлечения преподавателя для проверки каждого проекта индивидуально, а также расширить перечень решаемых задач за счёт нетиповых и открытых постановок.

Для апробации предложенных инструментов разрабатывается курс по классическому машинному обучению, охватывающий базовые разделы дисциплины: линейные модели, решающие деревья, ансамблевые методы и обучение без учителя. Структура курса предусматривает домашние задания в формате Jupyter Notebook с ячейками кода, открытыми теоретическими вопросами и итоговым проектом. Апробация проводится на студентах НИУ ВШЭ, что позволяет оценить работу системы в условиях реального учебного процесса.

ГЛАВА 2

2.1 Рабочая программа дисциплины

2.1.1 Пререквизиты

Для успешного освоения курса студент должен владеть:

– основами программирования на Python (синтаксис, функции, классы, работа с библиотеками);
– линейной алгеброй (матрицы, векторы, операции над ними);
– математическим анализом (производные, градиент);
– основами теории вероятностей и математической статистики.

2.1.2 Разделы

Курс состоит из восьми тематических разделов, охватывающих основные направления классического машинного обучения:

1) Введение в машинное обучение. Инструменты: NumPy, Pandas.
2) Линейная регрессия и методы оптимизации.
3) Классификация: логистическая регрессия, метод опорных векторов.
4) Деревья решений.
5) Ансамблевые методы: случайный лес, градиентный бустинг.
6) Кластеризация.
7) Специализированная тема по выбору: обработка естественного языка, компьютерное зрение или рекомендательные системы.
8) Консультация по проекту и защита проектов.

2.1.3 Темы

В рамках каждого раздела рассматриваются следующие темы:

– Разведочный анализ данных, предобработка, feature engineering, работа с дисбалансом классов.
– Математические основы машинного обучения: функции потерь, методы оптимизации (градиентный спуск и его модификации: SGD, Momentum, Adam).
– Линейные модели: линейная и логистическая регрессия, метод наименьших квадратов.
– Метрики качества классификации и регрессии: Precision, Recall, F1-мера,

ROC-AUC, PR-AUC, MSE, RMSE, MAE.
- Метод опорных векторов и ядровой трюк.
- Деревья решений для регрессии и классификации: критерии разбиения, переобучение, регуляризация.
- Ансамблевые методы: бэггинг, случайный лес, градиентный бустинг, bias-variance trade-off.
- Методы кластеризации и уменьшение размерности.

2.1.4 Оцениваемые задания

2.1.4.1 Домашние задания

На курсе предусмотрено три домашних задания, каждое рассчитано на две недели выполнения. Дедлайны строгие; работы, поданные после срока, не принимаются. Итоговая оценка за домашние задания — среднее арифметическое трёх работ.

- Домашнее задание 1. Работа с NumPy: векторизованные вычисления, операции над массивами.
- Домашнее задание 2. Градиентный спуск. Реализация с нуля классов VanillaGradientDescent, StochasticGradientDescent, MomentumDescent, Adam, а также класса LinearRegression с поддержкой L1/L2-регуляризации. Все реализации должны быть векторизованы. С использованием разработанных классов необходимо обучить модель, предсказывающую выбросы CO2. Датасет был самостоятельно сформирован из открытых источников. За прохождение определенных порогов целевой метрики студенты получали дополнительные баллы.
- Домашнее задание 3. Классификация текста. Предобработка текста, построение признаковых представлений, обучение и настройка классических моделей (логистическая регрессия, SVM), анализ метрик классификации и интерпретация результатов на задаче определения автора текста (человек или нейросеть). Домашнее задание частично выполняется в формате соревнования — наивысшую оценку (10 баллов) могут получить только топ-10 студентов, получивших максимальное значение целевой метрики на валидационной выборке.

2.1.4.2 Активность на семинарах

В ходе каждого семинара студентам предлагаются бонусные задания и ответы на вопросы. За одно занятие можно получить не более четырёх отметок активности. Итоговая оценка за активность — среднее за все занятия семестра. Шкала перевода:

– 4 отметки — 10 баллов;
– 3 отметки — 7,5 балла;
– 2 отметки — 5 баллов;
– 1 отметка — 2,5 балла;
– 0 отметок — 0 баллов.

Вопросы на семинарах формулировались так, чтобы подтолкнуть студентов к самостоятельному выводу, не давая ответ напрямую. Примеры вопросов: влияние коэффициента регуляризации λ на поведение модели при его уменьшении или росте; причины отказа от масштабирования OneHot-закодированных признаков; интерпретация метрики R^2.

Часть заданий носила практический характер: студентам предлагалось написать код для обучения модели или провести эксперименты с гиперпараметрами — например, исследовать влияние параметров случайного леса на качество модели и OOB-score.

2.1.4.3 Проект

Проект выполняется индивидуально (командные работы допускаются в исключительных случаях по согласованию с преподавателем). Работа ведётся в репозитории GitHub Classroom и сдаётся поэтапно через четыре чекпоинта с последующей защитой.

Требования к проекту:

– датасет — не менее 10 000 строк и не менее 9 признаков;
– наличие чётко сформулированной задачи машинного обучения;
– уникальность: датасет не должен пересекаться с проектами других студентов;
– при использовании Kaggle — только задачи типа monetary, задачи Getting Started не допускаются.

Структура сдачи и максимальные баллы:

– CP0 (выбор проекта, регистрация в GitHub Classroom) — допускной этап;
– CP1 (предобработка данных, базовое моделирование, качество кода) — 25 баллов, проходной порог 13 баллов;
– CP2 (расширенное моделирование, перебор гиперпараметров, ансамбли) — 30 баллов, проходной порог 15 баллов;
– CP3 (отчёт, деплой через FastAPI или аналог) — 30 баллов, проходной порог 15 баллов;
– защита (презентация 5 минут, ответы на вопросы) — 15 баллов.

Не набравший проходной балл на текущем чекпоинте не допускается к следующему; итоговая оценка за проект формируется из уже накопленных баллов.

2.1.5 Результаты обучения

По итогам курса студент:

– знает математические основы классических алгоритмов машинного обучения: линейные модели, деревья решений, ансамблевые методы, методы кластеризации;
– умеет реализовывать алгоритмы оптимизации (градиентный спуск и его модификации) с нуля на Python с использованием NumPy;
– умеет проводить полный цикл работы с данными: разведочный анализ, предобработку, feature engineering, построение и отбор моделей;
– умеет выбирать и обосновывать метрики качества для задач регрессии и классификации;
– имеет практические навыки работы с текстовыми данными и признаковыми представлениями;
– умеет организовывать воспроизводимый ML-проект: структура кода, линтеры, фиксация зависимостей;

2.2 Внедрение автоматизированной проверки в nbgrader

В стандартной версии nbgrader для ячеек с кодом доступны следующие типы: Autograded answer - ячейка, в которой студент пишет код, проверяемый автоматическими assert-тестами; Autograded tests - ячейка с тестами, скрытая от студента и запускаемая при проверке; Manually graded answer - ячейка с кодом или текстом, требующая ручной оценки; Manually graded task - аналогичный тип, но оценка выставляется на уровне задания целиком; Read-

only - ячейка, недоступная для редактирования студентом. Для текстовых (Markdown) ячеек набор типов ограничен: Manually graded answer, Manually graded task и Read-only. Таким образом, автоматическая проверка предусмотрена только для кода, а текстовые ответы могут быть оценены исключительно вручную.

2.2.1 Требования к работе

На основании проведенного анализа были сформулированы следующие требования к разрабатываемому решению.

Требования к новому типу ячейки:

1) В nbgrader должен быть добавлен новый тип ячейки - LLM graded answer, доступный для выбора наряду со стандартными типами.
2) Ячейка должна поддерживать структурированную разметку с разделителями: блок с формулировкой задания (видимый студенту) и блок с эталонным ответом и критериями оценивания (скрытый от студента).
3) Разделители должны быть настраиваемыми через конфигурацию.

Требования к генерации студенческой версии:

1) При генерации студенческой версии задания блок с критериями должен автоматически удаляться и заменяться текстом-заглушкой.
2) Формулировка задания должна сохраняться в неизменном виде.
3) Генерация должна выполняться стандартной командой nbgrader без дополнительных утилит.

Требования к конфигурации и промпту:

1) Преподаватель должен иметь возможность задать параметры подключения к API языковой модели: ключ, адрес, название модели.
2) Должен поддерживаться настраиваемый шаблон промпта с подстановкой переменных: формулировка задания, ответ студента, вывод ячейки, критерии оценивания, максимальный балл.
3) Должен быть задан таймаут ожидания ответа от API.

Требования к выходу модели и интеграции:

1) Модель должна возвращать численную оценку в диапазоне от 0 до максимального балла за задание.
2) Проверка ячеек типа LLM graded answer должна запускаться стандартной командой nbgrader autograde совместно с проверкой остальных типов

ячеек.

3) Результаты проверки должны сохраняться в базу данных nbgrader, обеспечивая совместимость с существующим интерфейсом просмотра оценок.

2.2.2 Реализация расширения nbgrader

В рамках работы в инструмент nbgrader был добавлен новый тип ячейки - LLM graded answer, предназначенный для заданий, ответ на которые требует проверки языковой моделью. На рис. 2.1 показан пример выбора типа ячейки с добавленным вариантом.

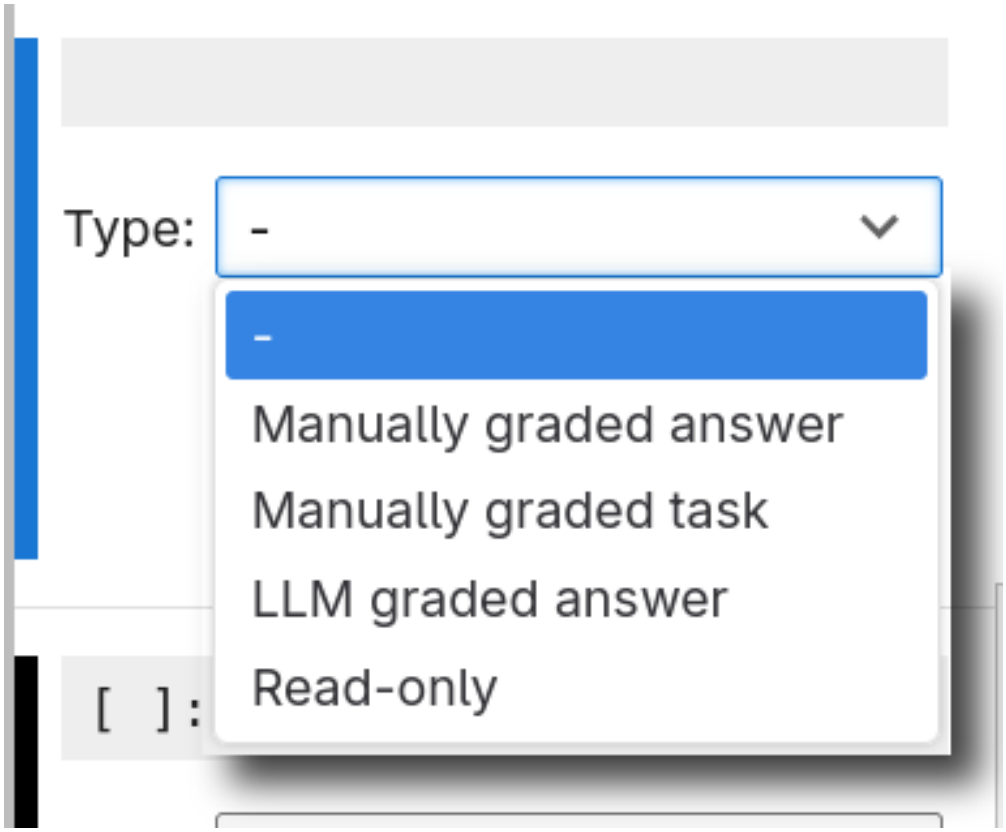

Рисунок 2.1 — Типы ячеек в nbgrader

2.2.3 Создание задания преподавателем

При подготовке преподавательской версии задания в Jupyter Notebook преподаватель выбирает тип ячейки LLM graded answer и заполняет её с помощью специальных разделителей, принятых в nbgrader:

– блок BEGIN QUESTION_LLM ... END QUESTION_LLM содержит формулировку задания, видимую студенту;
– блок BEGIN CRITERIA_LLM ... END CRITERIA_LLM содержит эталонный ответ и критерии оценивания, скрытые от студента.

Пример разметки ячейки приведён на рис. 2.2.

Задание 2.1. Градиент MSE в матричном виде (0.04 балла).

BEGIN QUESTION_LLM

Напомним, что функция потерь MSE записывается как:

$$Q(w) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \langle x_i, w\rangle)^2 = \frac{1}{n}\|Xw - y\|^2$$

где n – количество объектов в выборке, $X \in \mathbb{R}^{n\times d}$ – матрица "объект-признак", а $y \in \mathbb{R}^n$ – целевая переменная. Через x_i обозначается i-ая строчка матрицы X, отвечающая за i-й объект выборки.

Выпишите ниже (подсмотрев в семинар или решив самостоятельно) градиент для функции потерь MSE в матричном виде.

END QUESTION_LLM

BEGIN CRITERIA_LLM

Эталонный ответ: Градиент MSE в матричном виде: $\|Xw - y\|^2$ по w: $2X^\top(Xw - y)/n$.

Критерии оценивания (4 балла):

- 4 балла: указана правильная формула $2X^\top(Xw - y)/n$ в различных записях (latex, markdown)
- 0 баллов: неверная формула или отсутствие ответа.

END CRITERIA_LLM

Рисунок 2.2 — Пример ячейки, заполненной преподавателем

2.2.4 Генерация студенческой версии

Генерация студенческой версии задания выполняется стандартной командой nbgrader: nbgrader generate_assignment.

В результате блок с критериями удаляется, а на его месте появляется текст YOUR ANSWER HERE, сохраняя при этом формулировку задания в неизменном виде. Пример отображен на рис. 2.3.

Задание 2.1. Градиент MSE в матричном виде (0.04 балла). ¶

Напомним, что функция потерь MSE записывается как:

$$Q(w) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \langle x_i, w\rangle)^2 = \frac{1}{n}\|Xw - y\|^2$$

где n – количество объектов в выборке, $X \in \mathbb{R}^{n\times d}$ – матрица "объект-признак", а $y \in \mathbb{R}^n$ – целевая переменная. Через x_i обозначается i-ая строчка матрицы X, отвечающая за i-й объект выборки.

Выпишите ниже (подсмотрев в семинар или решив самостоятельно) градиент для функции потерь MSE в матричном виде.

YOUR ANSWER HERE

Рисунок 2.3 — Пример ячейки, которую видит студент

2.2.5 Конфигурация и запуск проверки

Перед запуском автоматической проверки преподаватель заполняет конфигурационный файл, задающий параметры обращения к БЯМ. Ключевые поля конфигурации:

- llm_api_key, llm_api_base, llm_model - параметры подключения к API языковой модели;
- БЯМ_prompt_template - шаблон промпта, в который подставляются:
  - {question} - формулировка задания;
  - {student_answer} - ответ студента;
  - {cell_output} - вывод ячейки (если есть);
  - {grading_instructions} - критерии оценивания;
  - {max_points} - максимальный балл за задание.
- begin_question_delimiter, end_question_delimiter и аналогичные поля для критериев - настройка разделителей разметки; по умолчанию равны BEGIN QUESTION_LLM и END QUESTION_LLM для заданий и BEGIN CRITERIA_LLM и END CRITERIA_LLM для критериев.
- llm_timeout - максимальное время ожидания ответа от API (в секундах).

Модель получает на вход перечисленные данные и возвращает число от 0 до max_points, представляющее выставленный балл.

После получения ответов студентов преподаватель запускает проверку стандартной командой: nbgrader autograde.

Система автоматически обходит все ячейки типа LLM graded answer, отправляет соответствующие запросы к языковой модели и сохраняет выставленные баллы в базу данных nbgrader. Остальные типы ячеек проверяются штатными механизмами инструмента.

2.3 Создание системы для проверки проектов

ГЛАВА 3

ЗАКЛЮЧЕНИЕ

В заключении подведены итоги проведённого исследования. Разработанная методика применения алгоритмов глубокого обучения для классификации данных показала свою эффективность на тестовых наборах. Проведённый анализ литературы и экспериментальные исследования подтвердили, что предложенный подход позволяет снизить процент ошибок классификации по сравнению с традиционными методами.

Основные выводы работы:

– разработанная методика оправдана с точки зрения теоретических обоснований и практического применения;
– экспериментальные результаты демонстрируют преимущества глубоких нейронных сетей для задач классификации;
– работа содержит рекомендации для дальнейших исследований и внедрения метода в информационные системы.

Таким образом, проведённое исследование подтверждает перспективность использования методов глубокого обучения в решении актуальных задач обработки данных.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Соколов Е. А. Машинное обучение 1. Курс факультета компьютерных наук НИУ ВШЭ. — дата обращения: 14.04.2026. URL: https://github.com/esokolov/ml-course-hse.
2. Курс машинного обучения факультета ВМК МГУ. — дата обращения: 14.04.2026. URL: https://github.com/MSU-ML-COURSE.
3. Оценивание курса ВМК МГУ. — дата обращения: 15.04.2026. URL: https://github.com/MSU-ML-COURSE/ML-COURSE-25-26/blob/master/slides/intro/ML$_g rading_s ystem.pdf$.
4. Deep Learning School при ФПМИ МФТИ. — дата обращения: 14.04.2026. URL: https://dls.samcs.ru/.
5. Open ML Course: Классические модели ML. — дата обращения: 15.04.2026. URL: https://ods.ai/tracks/classic-models-spring26.
6. Открытый курс машинного обучения girafe-ai. — дата обращения: 14.04.2026. URL: https://github.com/girafe-ai/ml-course.
7. Karpov.Courses: Инженер машинного обучения. — дата обращения: 14.04.2026. URL: https://karpov.courses/ml-start.
8. Яндекс Практикум: ML-инженер с опытом. — дата обращения: 18.04.2026. URL: https://practicum.yandex.ru/machine-learning/.
9. nbgrader: A system for assigning and grading notebooks. — дата обращения: 14.04.2026. URL: https://nbgrader.readthedocs.io/en/stable/.
10. nbgrader: Github repository. — дата обращения: 14.04.2026. URL: https://github.com/jupyter/nbgrader.
11. Otter-Grader: A Python and R autograding solution. — дата обращения: 14.04.2026. URL: https://otter-grader.readthedocs.io/en/latest/.
12. Otter-Grader: Github repository. — дата обращения: 14.04.2026. URL: https://github.com/ucbds-infra/otter-grader.
13. CodeGrade: The Engaging Code Learning Platform. — дата обращения: 14.04.2026. URL: https://www.codegrade.com/.

14. Яндекс Контест — платформа для решения задач по программированию. — дата обращения: 14.04.2026. URL: https://contest.yandex.ru/.
15. Stepik: образовательная онлайн-платформа. — дата обращения: 14.04.2026. URL: https://stepik.org/.
16. Use autograding — GitHub Classroom Documentation. — дата обращения: 15.04.2026. URL: https://docs.github.com/en/education/manage-coursework-with-github-classroom/teach-with-github-classroom/use-autograding.
17. Kaggle Competitions. — дата обращения: 15.04.2026. URL: https://www.kaggle.com/competitions.
18. PyEvalAI: AI-assisted evaluation of Jupyter Notebooks for immediate personalized feedback. — дата обращения: 14.04.2026. URL: https://arxiv.org/abs/2502.18425.

ПРИЛОЖЕНИЕ. А РУКОВОДСТВО ПО ИСПОЛЬЗОВАНИЮ ШАБЛОНА

В данном приложении приведены примеры оформления основных элементов ВКР согласно требованиям ИТМО (версия 4.0, 2022 г.).

Пример заголовков разделов и подразделов

Ниже показаны три допустимых варианта оформления заголовков (п. 4.4.4 требований ИТМО).

Вариант 1	**Вариант 2**	**Вариант 3**
1 НАЗВАНИЕ РАЗДЕЛА **1.1 Название подраздела** 1.1.1 Название подпункта 1.1.2 Название подпункта	**1 Название раздела** **1.1 Название подраздела** 1.1.1 Название подпункта 1.1.2 Название подпункта	1 Название раздела 1.1 Название подраздела 1.1.1 Название подпункта 1.1.2 Название подпункта

Правила оформления заголовков:

– заголовки разделов и подразделов начинаются с абзацного отступа,
– в конце заголовка точка не ставится,
– переносы слов в заголовках не допускаются,
– аббревиатуры в заголовках не допускаются.

Пример изображения

Ссылка на рисунок обязательна в тексте перед его вставкой (рисунок А.1):

Примеры таблиц

Ссылка на таблицу обязательна перед её вставкой (таблица А.1):

Таблица А.1 — Пример построения таблицы

Заголовок	**Заголовок колонки**		**Заголовок колонки**
	Подзаголовок	**Подзаголовок**	
A	B	C	D

Пример таблицы с переносом на следующую страницу (таблица А.2):

Рисунок А.1 — Пример изображения, вставленного в приложение

Таблица А.2 — Пример переноса таблицы

Заголовок	**Заголовок колонки**	**Заголовок колонки**	**Заголовок колонки**
1	2	3	4
2	3	4	5

Продолжение таблицы А.2

Заголовок	**Заголовок колонки**	**Заголовок колонки**	**Заголовок колонки**
3	4	5	6
4	5	6	7

Примеры перечислений

Вариант 1 — маркированный список через тире (п. 4, Приложение 1 требований ИТМО):

Информационно-сервисная служба для обслуживания удалённых пользователей включает следующие модули:

– удалённый заказ,
– виртуальная справочная служба,

– виртуальный читальный зал.

Вариант 2 — нумерованный список с цифрой и скобкой:

Работа по оцифровке включала следующие технологические этапы:

1) первичный осмотр и структурирование исходных материалов,
2) сканирование документов,
3) обработка и проверка полученных образов,
4) выходной контроль качества.

Вариант 3 — многоуровневый список:

Разрабатываемое устройство можно применять в различных отраслях:

– в машиностроении:
  1) для очистки отливок от формовочной смеси,
  2) для холодной штамповки из листа;
– в ремонте техники:
  1) устранение наслоений на внутренних стенках труб,
  2) очистка каналов небольшого диаметра.

Пример формулы

Функция потерь определяется формулой (А.1):

$$L(y, \hat{y}) = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2, \tag{А.1}$$

где y_i — истинное значение, $\hat{y}_i$ — предсказанное значение, n — объём выборки.

Инструкция: Приложение содержит дополнительные материалы, не вошедшие в основной текст работы. В него можно включать изображения, таблицы, программный код, результаты экспериментов и другие вспомогательные данные.